SYSTEM DESIGN KNOWLEDGE LIBRARY

From Beginner to Principal Architect

Document Overview

This comprehensive library contains a complete taxonomy of System Design concepts organized across 50 major domains. It spans from foundational engineering concepts through advanced distributed systems, with specialized domains for cloud architecture, AI/ML systems, and more.

The curriculum is designed to support three learning dimensions:

- Knowledge Depth: Concept → Subconcept → Advanced Concept
- Difficulty Level: Beginner → Intermediate → Advanced → Expert → Principal Architect
- Application Method: Learn → Visualize → Simulate → Design → Implement → Break → Optimize → Interview

Table of Contents

- Level 0: Engineering Foundations
- Level 1: Web & Internet Fundamentals
- Level 2: API & Backend Fundamentals
- Level 3: Databases
- Level 4: NoSQL Databases
- Level 5: Caching
- Level 6: Load Balancing & Traffic Management
- Level 7: CDN & Edge Computing
- Level 8: Message Queues & Asynchronous Systems
- Level 9: Event-Driven Architecture
- Levels 10-50: Advanced Domains
- Case Studies & Learning Paths
- Implementation Framework

LEVEL 0: ENGINEERING FOUNDATIONS

0.1 Computing Fundamentals

Understanding the basics of how computers and applications operate

- Client and Server
- Application, Process, and Thread
- CPU, Memory, and Storage Management
- Network Basics
- Operating System and Kernel
- File System
- Runtime and Application Lifecycle
- Resource Utilization Principles

0.2 How Applications Work

- Request/Response Model
- Application Flow
- Input/Output Operations
- State Management
- Stateless vs. Stateful Applications

- Session Handling
- Authentication and Authorization
- Configuration Management
- Environment Variables
- Logging Strategies

0.3 Basic Architecture Patterns

- Monolithic Architecture
- Modular Monolith
- Layered Architecture
- Three-Tier and N-Tier Patterns
- Client-Server Architecture
- Service-Oriented Architecture
- Introduction to Microservices

LEVEL 1: WEB & INTERNET FUNDAMENTALS

1.1 Internet Fundamentals

- IP Addresses (IPv4, IPv6)
- Ports and Port Forwarding
- TCP vs. UDP Protocols
- Packet Transmission
- Routing Mechanisms
- NAT (Network Address Translation)
- Firewalls and Security

1.2 DNS (Domain Name System)

- DNS Resolution Process
- Recursive vs. Authoritative DNS
- DNS Records (A, AAAA, CNAME, MX, TXT, NS)
- TTL (Time To Live)
- DNS Caching
- DNS Propagation
- GeoDNS and Load Balancing

1.3 HTTP Protocol

- HTTP Request/Response Cycle
- HTTP Methods (GET, POST, PUT, PATCH, DELETE)
- HTTP Headers
- Status Codes
- Cookies and Sessions
- HTTP Caching Strategies
- Compression (gzip, brotli)
- HTTP/1.1, HTTP/2, HTTP/3, and QUIC

1.4 HTTPS & TLS

- SSL vs. TLS
- Certificate Authorities
- Public/Private Key Cryptography
- TLS Handshake Process
- Encryption Algorithms
- Certificate Management

- HSTS (HTTP Strict Transport Security)

LEVEL 2: API & BACKEND FUNDAMENTALS

2.1 API Design Principles

- REST Principles and Constraints
- Resource Modeling
- URI Design Best Practices
- HTTP Semantics
- API Versioning Strategies
- Pagination, Filtering, and Sorting
- Error Handling
- Partial Responses and Content Negotiation

2.2 API Styles & Protocols

- REST APIs
- GraphQL
- gRPC
- WebSockets
- Server-Sent Events
- Webhooks
- SOAP and RPC

2.3 API Reliability Patterns

- Idempotency and Request Deduplication
- Request Timeouts
- Retry Policies and Exponential Backoff
- Rate Limiting and Throttling
- Circuit Breaker Pattern
- Bulkhead Isolation
- Request Validation

2.4 API Gateway

- API Gateway Architecture
- Reverse Proxy Functionality
- Routing and Service Discovery
- Authentication and Authorization
- Request/Response Transformation
- API Aggregation
- Gateway Security

LEVEL 3: DATABASES

3.1 Database Fundamentals

- Tables, Rows, and Columns
- Primary Keys and Foreign Keys
- Constraints
- Indexes
- ACID Properties
- CRUD Operations

3.2 Relational Databases

- SQL Query Language
- Popular Systems (PostgreSQL, MySQL, SQL Server, Oracle)
- Joins (INNER, LEFT, RIGHT, FULL)
- Aggregations and Grouping
- Transaction Management
- Normalization Forms

3.3 Database Design

- Entity-Relationship Modeling
- One-to-One, One-to-Many, Many-to-Many Relationships
- Normalization vs. Denormalization
- Schema Design
- Key Selection (Composite vs. Surrogate)
- Data Modeling Best Practices

3.4 Indexing & Query Optimization

- Index Fundamentals
- B-Tree and Hash Indexes
- Composite and Covering Indexes
- Clustered vs. Non-Clustered Indexes
- Index Selectivity
- Query Planning and Optimization

3.5 Transactions & Consistency

- ACID Properties
- Isolation Levels
- Read Uncommitted, Read Committed, Repeatable Read, Serializable
- Dirty Reads, Non-Repeatable Reads, Phantom Reads
- Lock Management
- Deadlock Detection and Prevention

3.6 Database Scaling

- Vertical vs. Horizontal Scaling
- Read Replicas
- Replication Strategies
- Database Sharding
- Consistent Hashing
- Shard Key Selection
- Cross-Shard Queries

3.7 Distributed Databases

- Leader/Follower Architectures
- Multi-Leader Replication
- Leaderless Systems
- Quorum-Based Consistency
- Eventual vs. Strong Consistency
- Distributed Transactions

LEVELS 4-50: ADVANCED DOMAINS

The complete taxonomy extends across 50 major domains covering advanced system design concepts:

Data Storage & Retrieval

- Level 4: NoSQL Databases (Key-Value, Document, Wide-Column, Graph)
- Level 5: Caching (Strategies, Eviction Policies, Distributed Caching)
- Level 23: Modern Data Architecture (Data Lakes, Data Warehouses, Data Mesh)

Traffic & Infrastructure

- Level 6: Load Balancing & Traffic Management
- Level 7: CDN & Edge Computing
- Level 35-37: Cloud & Global Architecture

Asynchronous & Event Processing

- Level 8: Message Queues & Asynchronous Systems
- Level 9: Event-Driven Architecture
- Level 43: Real-Time Systems

Distributed Systems & Architecture

- Level 10: Distributed Systems Fundamentals
- Level 11: Replication & Data Distribution
- Level 12: Microservices Architecture
- Level 13: API Design & Integration
- Level 14: Search Systems

Reliability & Operations

- Level 15: Monitoring, Logging & Observability
- Level 16: Distributed Tracing
- Level 30: Reliability Engineering
- Level 31: Observability
- Level 32: DevOps & Platform Engineering

Security & Compliance

- Level 17: Security
- Level 18: Data Security & Encryption
- Level 38: Security Architecture
- Level 39: Governance

Domain-Specific Systems

- Level 19: Notification Systems
- Level 20: Background Jobs
- Level 21: Rate Limiting & Quotas
- Level 22: Configuration Management
- Level 40: AI/ML Architecture
- Level 44: Geospatial Systems
- Level 45: Recommendation Systems

Deployment & Operations

- Level 24: Service Mesh
- Level 25: Containerization & Orchestration
- Level 26: Infrastructure as Code
- Level 27: Container Networking
- Level 28: Secrets Management
- Level 33: Deployment & Release Engineering

- Level 34: Disaster Recovery

Emerging & Specialized

- Level 29: Testing & Chaos Engineering
- Level 41: Sustainability Engineering
- Level 42: Cost & FinOps
- Level 46: Architecture Migration
- Level 47: System Design Testing & Chaos

REAL-WORLD CASE STUDIES

Case studies are organized by difficulty level, each demonstrating how multiple domains interact in practice:

Beginner Level

- URL Shortener - Learn: DNS, HTTP, databases, basic caching
- Pastebin - Learn: File storage, text search, user authentication
- Simple Chat - Learn: WebSockets, real-time communication, persistence
- Notification System - Learn: Events, message queues, delivery reliability

Intermediate Level

- Instagram - Learn: Photo storage, feed generation, sharding strategies
- Twitter/X - Learn: Timelines, graph models, geo-distribution
- WhatsApp - Learn: Real-time messaging, presence, reliability at scale
- YouTube - Learn: Video streaming, CDN, adaptive bitrate, search indexing
- Netflix - Learn: Transcoding, CDN strategy, failure handling
- Uber - Learn: Geospatial indexing, real-time tracking, matching algorithms
- Food Delivery - Learn: Location-based search, routing, notifications

Advanced Level

- Google Search - Learn: Web crawling, indexing, ranking, distributed query
- Google Maps - Learn: Geospatial data, navigation, real-time traffic
- Distributed Cache - Learn: Consistency, cache invalidation, partitioning
- Payment Gateway - Learn: Distributed transactions, consistency, PCI compliance

Expert/Principal Level

- Global Payment System - Learn: Multi-region consistency, settlement, fraud detection
- Global Video Platform - Learn: Multi-region CDN, transcoding at scale, monetization
- AI Inference Platform - Learn: Model serving, auto-scaling, batching, GPU scheduling
- Global Observability Platform - Learn: Metrics aggregation, log processing at scale

LEARNING FRAMEWORK

The Codorythm Three-Dimensional Learning Model

Effective system design learning requires engagement across three dimensions:

Dimension	Progression
Knowledge Depth	Concept → Subconcept → Advanced Concept
Difficulty Level	Beginner → Intermediate → Advanced → Expert → Principal Architect
Application Method	Learn → Visualize → Simulate → Design → Implement → Break → Optimize → Int

Engagement Methodology

- Learn: Read comprehensive concept documentation with examples
- Visualize: Interactive diagrams showing how components interact
- Simulate: Run scenarios to understand behavior (e.g., cache eviction, shard distribution)
- Design: Apply concepts to design a complete system
- Implement: Write code to build the system
- Break: Introduce failures (hot shards, network partitions) to test resilience
- Optimize: Improve for scalability, reliability, cost, and consistency
- Interview: Get evaluated by AI interviewer on scalability, reliability, cost, and consistency

RECOMMENDED LEARNING PATHS

Path 1: Complete Beginner to Senior Engineer (6-12 months)

- Months 1-2: Levels 0-2 (Foundations, Web, APIs)
- Months 3-4: Level 3 (Relational Databases)
- Months 5-6: Levels 4-5 (NoSQL, Caching)
- Months 7-8: Levels 6-9 (Load Balancing, CDN, Messages, Events)
- Months 9-10: Levels 10-12 (Distributed Systems, Microservices)
- Months 11-12: Case Studies (Beginner → Intermediate)
- Capstone: Design 3-4 intermediate systems under mentorship

Path 2: Senior to Staff Engineer (8-16 months)

- Months 1-2: Levels 13-18 (Specialized domains)
- Months 3-4: Levels 19-25 (Notifications, Jobs, Service Mesh, Containers)
- Months 5-6: Levels 26-31 (Infrastructure, DevOps, Reliability)
- Months 7-8: Levels 32-35 (Deployment, Disaster Recovery, Cloud)
- Months 9-12: Levels 36-42 (Global Architecture, Security, FinOps)
- Months 13-16: Levels 43-50 (Real-Time, Data, AI, Migration, Chaos)
- Capstone: Design 2-3 advanced systems, mentor juniors

Path 3: Staff to Principal Architect (12-24 months)

- Phase 1 (4 months): Deep dives into expert case studies
- Phase 2 (4 months): Architecture decision frameworks and trade-off analysis
- Phase 3 (4 months): System design at massive scale (planet-scale systems)
- Phase 4 (4 months): Leadership, mentoring, and organization-wide architecture
- Phase 5 (4 months): Industry research and emerging technologies
- Phase 6 (4 months): Author architecture patterns and contribute to industry
- Capstone: Design a revolutionary system that sets new industry standards

KEY LEARNING OUTCOMES BY LEVEL

Beginner Level

- ✓ Understand client-server architecture
- ✓ Design basic 3-tier applications
- ✓ Work with SQL databases
- ✓ Implement REST APIs
- ✓ Use basic caching strategies
- ✓ Deploy to single server

Intermediate Level

- ✓ Design for 1M concurrent users
- ✓ Implement database sharding
- ✓ Build microservices
- ✓ Use message queues
- ✓ Implement event-driven systems
- ✓ Deploy to Kubernetes
- ✓ Design distributed systems

Advanced Level

- ✓ Design for 100M+ users
- ✓ Multi-region deployment
- ✓ Global data consistency models
- ✓ Advanced search and recommendation
- ✓ Real-time systems at scale
- ✓ Machine learning systems
- ✓ Cost optimization strategies

Expert/Principal Level

- ✓ Design planet-scale systems
- ✓ Set industry standards
- ✓ Mentor teams and architects
- ✓ Define architectural governance
- ✓ Drive technology strategy
- ✓ Research emerging technologies
- ✓ Author industry-leading designs

CONCLUSION & NEXT STEPS

This System Design Knowledge Library provides a comprehensive framework for mastering distributed systems and architecture design. The 50-domain taxonomy, combined with the three-dimensional learning model, creates a structured path from beginner fundamentals through principal-level expertise.

Key Principles for Success:

- Progressive Mastery: Each level builds on previous knowledge. Don't skip foundations.
- Hands-On Learning: Theory alone is insufficient. Simulate, implement, and break systems.
- Understand Trade-offs: Every design decision involves trade-offs between consistency, availability, scalability, and cost.
- Learn from Real Systems: Case studies show how theory applies to production systems.
- Stay Current: Technology evolves. Keep up with emerging patterns and tools.

Recommended Starting Points:

- For Students: Start with Level 0-2, then URL Shortener case study
- For Junior Engineers: Complete Levels 0-9, then Intermediate case studies
- For Senior Engineers: Focus on Levels 10-25 and Advanced case studies
- For Staff/Principal: Study Levels 26-50 and Expert case studies

Remember: System design mastery is not about memorizing solutions. It's about developing intuition for how systems behave at scale and making informed architectural decisions.